CO2-Assignment-2

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: CSA0601

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

INTRODUCTION

An algorithm is a step-by-step procedure used to solve a problem. In computer science, algorithms must produce correct results while using reasonable execution time and memory. The Design and Analysis of Algorithms subject helps us understand how algorithms behave when the input size increases.

This assignment focuses on sorting, searching, string matching, computational geometry, and exhaustive search. Brute-force algorithms solve problems by checking possible candidates directly. They are simple to understand and implement, but their execution time may increase significantly for large inputs.

Exhaustive search examines every possible solution and selects the best valid solution. It can guarantee an optimal result when all possibilities are checked correctly, but it becomes expensive when the number of possible solutions grows exponentially.

The main objective of this assignment is to implement different algorithms, predict their behaviour, measure their operation counts, analyse their time complexity, and identify situations where an improved algorithm is required.

Objectives

- To study sorting algorithms under different input conditions.
- To measure the comparisons performed by sequential search.
- To analyse brute-force string matching using different text-pattern pairs.
- To derive the number of pairs examined by the closest-pair algorithm.
- To implement a brute-force convex-hull algorithm.
- To understand exhaustive search through a humanitarian airlift problem.
- To identify an algorithmic bottleneck and propose an improved solution.

Q1. SORTING UNDER DIFFERENT DATA CONDITIONS

1. Problem Context

An e-commerce fulfilment centre needs to arrange customer orders according to their dispatch-priority scores. Orders with higher priority must be processed earlier. The centre considers three sorting algorithms: Selection Sort, Bubble Sort, and Insertion Sort.

The performance of these algorithms depends on the initial arrangement of the input data. Therefore, different datasets are used to understand how each algorithm behaves.

2. Selection Sort

Selection Sort repeatedly finds the smallest element from the unsorted portion of the list and places it in the correct position.

The algorithm divides the list into two portions: the sorted portion and the unsorted portion. Initially, the sorted portion is empty. During each pass, the smallest element from the unsorted portion is selected and exchanged with the first unsorted element.

Time complexity: Best, average and worst cases are \(O(n^2)\).

Advantages:

- Simple to understand and implement.
- Requires only a small amount of additional memory.
- Performs relatively few swaps.

Limitations:

- Performs many comparisons even when the list is already sorted.
- Not suitable for large datasets when efficiency is important.

3. Bubble Sort

Bubble Sort compares adjacent elements and exchanges them when they are in the wrong order. This process continues until the list is sorted.

An early-stopping condition can be included. If no swaps occur during a complete pass, the algorithm stops because the list is already sorted.

Time complexity:

- Best case with early stopping: $O(n)$.
- Average case: $O(n^2)$.
- Worst case: $O(n^2)$.

Advantages:

- Easy to implement.
- Can perform efficiently on an already sorted list when early stopping is used.
- Requires $O(1)$ auxiliary space.

Limitations:

- Performs many swaps for reverse-ordered data.
- Becomes inefficient for large lists.

4. Insertion Sort

Insertion Sort builds a sorted list one element at a time. Each new element is compared with the previously sorted elements and inserted into its correct position.

It is especially useful when the input list is already sorted or nearly sorted.

Time complexity:

- Best case: $O(n)$.
- Average case: $O(n^2)$.
- Worst case: $O(n^2)$.

Advantages:

- Efficient for small datasets.
- Performs well with nearly sorted data.
- Requires $O(1)$ auxiliary space.
- Suitable when new elements are added to an existing sorted list.

Limitations:

- Requires many shifts when the list is in reverse order.

- Not suitable for very large, randomly arranged datasets.

5. Designed Test Datasets

Three datasets are used to compare the algorithms.

Dataset	Input values	Purpose
Nearly sorted	10, 20, 30, 25, 40, 50	Tests performance when most values are already ordered
Reverse ordered	60, 50, 40, 30, 20, 10	Tests performance on a difficult arrangement
All equal	5, 5, 5, 5, 5, 5	Tests duplicate values and unnecessary movements

The nearly sorted dataset represents the normal operating condition of the fulfilment centre. The reverse-ordered dataset represents a contrasting condition. The all-equal dataset is an edge case that checks whether the algorithms perform unnecessary movements.

6. Performance Predictions

Input condition	Selection Sort	Bubble Sort	Insertion Sort
Already sorted	Quadratic comparisons	Linear with early stopping	Linear
Nearly sorted	Quadratic comparisons	Usually fewer swaps	Usually fewer shifts
Reverse ordered	Quadratic comparisons	Many swaps	Many shifts
All equal	Quadratic comparisons	One pass with early stopping	Very few shifts

Selection Sort continues to perform quadratic comparisons regardless of the input arrangement. Bubble Sort can stop early when the list is sorted. Insertion Sort is efficient when only a few elements need to be moved.

7. Python Implementation

```
import time

def selection_sort(a):
    a = a.copy()
```

```
    comparisons = swaps = 0
    n = len(a)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]
            swaps += 1
    return a, comparisons, swaps
def bubble_sort(a):
    a = a.copy()
    comparisons = swaps = 0
    n = len(a)
    for i in range(n - 1):
        changed = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                changed = True
```

```
        if not changed:
            break
    return a, comparisons, swaps
def insertion_sort(a):
    a = a.copy()
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
                break
        a[j + 1] = key
    return a, comparisons, shifts
datasets = {
    "Nearly sorted": [10, 20, 30, 25, 40, 50],
    "Reverse sorted": [60, 50, 40, 30, 20, 10],
    "All equal": [5, 5, 5, 5, 5, 5]
}
```

```
algorithms = {
    "Selection": selection_sort,
    "Bubble": bubble_sort,
    "Insertion": insertion_sort
}
for name, data in datasets.items():
    print("\nDataset:", name)
    for alg_name, algorithm in algorithms.items():
        start = time.perf_counter()
        result, comparisons, movements = algorithm(data)
        elapsed = time.perf_counter() - start
        print(alg_name, "Sorted:", result,
              "Comparisons:", comparisons,
              "Swaps/Shifts:", movements,
              "Time:", round(elapsed, 8), "seconds")
```

OUTPUT:

```
Dataset: Nearly sorted
Selection Sorted: [10, 20, 25, 30, 40, 50] Comparisons: 15 Swaps/Shifts: 1 Time: 1.497e-05 seconds
Bubble Sorted: [10, 20, 25, 30, 40, 50] Comparisons: 9 Swaps/Shifts: 1 Time: 6.89e-06 seconds
Insertion Sorted: [10, 20, 25, 30, 40, 50] Comparisons: 6 Swaps/Shifts: 1 Time: 5.23e-06 seconds

Dataset: Reverse sorted
Selection Sorted: [10, 20, 30, 40, 50, 60] Comparisons: 15 Swaps/Shifts: 3 Time: 5.86e-06 seconds
Bubble Sorted: [10, 20, 30, 40, 50, 60] Comparisons: 15 Swaps/Shifts: 15 Time: 5.9e-06 seconds
Insertion Sorted: [10, 20, 30, 40, 50, 60] Comparisons: 15 Swaps/Shifts: 15 Time: 5.57e-06 seconds

Dataset: All equal
Selection Sorted: [5, 5, 5, 5, 5, 5] Comparisons: 15 Swaps/Shifts: 0 Time: 3.67e-06 seconds
Bubble Sorted: [5, 5, 5, 5, 5, 5] Comparisons: 5 Swaps/Shifts: 0 Time: 2.08e-06 seconds
Insertion Sorted: [5, 5, 5, 5, 5, 5] Comparisons: 5 Swaps/Shifts: 0 Time: 1.94e-06 seconds
```

8. Complexity Analysis

Algorithm	Best case	Average case	Worst case	Extra space

Selection Sort	\(O(n^2)\)	\(O(n^2)\)	\(O(n^2)\)	\(O(1)\)
Bubble Sort with early stopping	\(O(n)\)	\(O(n^2)\)	\(O(n^2)\)	\(O(1)\)
Insertion Sort	\(O(n)\)	\(O(n^2)\)	\(O(n^2)\)	\(O(1)\)

For Selection Sort, the total number of comparisons is

\[C(n)=\frac{n(n-1)}{2} \]

This expression demonstrates quadratic growth.

9. Results and Recommendation

The output of the program should be examined for each dataset. The comparison and movement counts help explain the behaviour of each algorithm. Execution time is also recorded, but it can vary depending on the computer and background processes.

Insertion Sort is recommended for the fulfilment centre because approximately 95% of the existing orders are already in the correct order and only small batches of new orders are added. Insertion Sort can place new orders into their correct positions without repeatedly sorting the entire list from the beginning.

Therefore, Insertion Sort is the most suitable choice among the three algorithms for this specific condition.

Q2 – When Is Sequential Search an Appropriate Engineering Choice?

1. Problem Context

An aircraft maintenance organisation maintains records of specialised tools issued to technicians. Each record contains the Tool ID, Technician ID, Issue Time and Work Bay.

When a technician requests a tool, the system must locate the corresponding record. Sequential Search can be used to search for a Tool ID in a list.

2. Working Principle

Sequential Search checks each element from the beginning of the list until the required element is found or the end of the list is reached.

For example, consider the list:

[T-101, T-184, T-220, T-310, T-450]

If the required Tool ID is T-101, only one comparison is required. If the required Tool ID is T-450, five comparisons are required. If the Tool ID is not present, every element must be checked.

3. Test Cases

Search case	**Search key**	**Expected comparisons**

First element	T-101	1
Middle element	T-220	3
Last element	T-450	5
Unsuccessful search	T-999	5

The number of comparisons depends on the location of the key and whether the key exists in the list.

4. Python Implementation

```
def sequential_search(records, key):
    comparisons = 0
    for index, record in enumerate(records):
        comparisons += 1
        if record == key:
            return index, comparisons
    return -1, comparisons
for n in [5, 10, 20, 100]:
    records = [f"T-{100 + i}" for i in range(n)]
    tests = [
        ("First", records[0]),
        ("Middle", records[n // 2]),
        ("Last", records[-1]),
        ("Missing", "T-999999")
    ]
    print("\nList size:", n)
    for name, key in tests:
        index, comparisons = sequential_search(records, key)
        print(name, "Index:", index,
              "Comparisons:", comparisons)
```

OUTPUT:

```
List size: 5
First Index: 0 Comparisons: 1
Middle Index: 2 Comparisons: 3
Last Index: 4 Comparisons: 5
Missing Index: -1 Comparisons: 5

List size: 10
First Index: 0 Comparisons: 1
Middle Index: 5 Comparisons: 6
Last Index: 9 Comparisons: 10
Missing Index: -1 Comparisons: 10

List size: 20
First Index: 0 Comparisons: 1
Middle Index: 10 Comparisons: 11
Last Index: 19 Comparisons: 20
Missing Index: -1 Comparisons: 20

List size: 100
First Index: 0 Comparisons: 1
Middle Index: 50 Comparisons: 51
Last Index: 99 Comparisons: 100
Missing Index: -1 Comparisons: 100
```

5. Complexity Analysis

The best-case time complexity is $O(1)$, because the key may be found in the first position.

The worst-case time complexity is $O(n)$, because the key may be in the last position or may not exist in the list.

For a successful search in which every position is equally likely, the average number of comparisons is

$$ C_{\text{average}}=\frac{n+1}{2} $$

The space complexity is $O(1)$, because the algorithm uses only a constant amount of additional memory.

6. Engineering Recommendation

Sequential Search is appropriate when the number of records is small, searches are infrequent, or the data changes frequently and maintaining an index would add unnecessary complexity.

However, if the organisation expands to 20 maintenance facilities, the number of tool records and lookup requests may increase significantly. Repeatedly scanning all records can then become inefficient.

A hash table or a database index can provide faster lookups. A hash table offers expected $O(1)$ lookup time, while a database index is suitable for persistent records and multiple users.

Therefore, Sequential Search is suitable for small collections, but an indexed search method is preferable when the system must support a large number of repeated queries.

Q3 – Engineer the Behaviour of Brute-Force String Matching

1. Problem Context

A security operations centre scans system logs to identify known event signatures. A signature is represented by a pattern string. The system must determine whether the pattern occurs in the text and identify its position.

Brute-Force String Matching checks the pattern at every possible position in the text.

2. Working Principle

Let the text length be \(n\) and the pattern length be \(m\). The algorithm aligns the pattern with the beginning of the text and compares characters from left to right.

If all characters match, the pattern has been found. If a mismatch occurs, the algorithm shifts the pattern one position to the right and repeats the comparison.

The process continues until all possible alignments have been checked.

3. Pseudocode

```
BRUTE_FORCE_MATCH(text, pattern)
    n = length(text)
    m = length(pattern)
    comparisons = 0
    for shift = 0 to n - m
        j = 0
        while j < m
            comparisons = comparisons + 1
            if text[shift + j] != pattern[j]
                break
            j = j + 1
        if j == m
            report match at shift
    return comparisons
```

4. Designed Test Cases

Test	Text	Pattern	Expected behaviour
Early match	abcdef	abc	Match occurs at the beginning

Repeated prefix	aaaaaaaaab	aaaab	Repeated partial matches cause additional comparisons
Immediate mismatch	abcdefgh	xyz	Each alignment fails quickly

These test cases are designed to demonstrate how character arrangement affects the number of comparisons.

5. Python Implementation

```
def brute_force_match(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0
    matches = []
    if m == 0:
        return list(range(n + 1)), comparisons
    if m > n:
        return matches, comparisons
    for shift in range(n - m + 1):
        j =
        while j < m:
            comparisons += 1
            if text[shift + j] != pattern[j]:
                break
            j += 1
        if j == m:
            matches.append(shift)
    return matches, comparisons
tests = [
    ("Early match", "abcdef", "abc"),
    ("Repeated prefix", "aaaaaaaaab", "aaaab"),
    ("Immediate mismatch", "abcdefgh", "xyz")
]
```

```
for name, text, pattern in tests:
    matches, comparisons = brute_force_match(text, pattern)
    print(name)
    print("Matches:", matches)
    print("Character comparisons:", comparisons)
```

OUTPUT:

```
Early match
Matches: [0]
Character comparisons: 6
Repeated prefix
Matches: [5]
Character comparisons: 30
Immediate mismatch
Matches: []
Character comparisons: 6
```

6. Analysis of Character Comparisons

In the early-match case, the algorithm finds the pattern at the beginning. Later positions are checked because the implementation is designed to find all occurrences.

In the repeated-prefix case, several alignments begin with the same character as the pattern. The algorithm compares multiple characters before finding a mismatch. This causes repeated work.

In the immediate-mismatch case, the first character of the pattern differs from the corresponding text character at each alignment. Therefore, the number of comparisons is relatively low.

These observations show that the number of comparisons depends not only on the text length but also on the arrangement of characters in the text and pattern.

7. Complexity Analysis

The worst-case time complexity is

\[O(nm) \]

where \(n\) is the text length and \(m\) is the pattern length.

The best case is \(O(n)\) when each possible alignment fails immediately. The auxiliary space complexity is \(O(1)\), excluding the list used to store matching positions.

8. Conclusion

Brute-Force String Matching is simple and suitable for short texts and patterns. However, it may repeat many character comparisons when the text and pattern contain repeated prefixes. For large log files or systems that search for many patterns, more advanced string-matching methods can reduce repeated work.

Q4 – Closest Pair and the Cost of Examining Every Possibility

1. Problem Context

A logistics company uses delivery drones to transport packages. Each drone reports its current position as a two-dimensional coordinate (x,y).

The monitoring system must identify the two drones that are closest to each other. The Brute-Force Closest Pair algorithm calculates the distance between every unique pair of points and selects the pair with the smallest distance.

2. Euclidean Distance

The Euclidean distance between two points $A=(x_1,y_1)$ and $B=(x_2,y_2)$ is

$$ d(A,B)=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2} $$

The algorithm calculates this distance for every unique pair of points.

3. Derivation of the Number of Pairs

For n points, the first point can be paired with $n-1$ other points. The second point can be paired with $n-2$ points that have not already been counted. This continues until only one pair remains.

Therefore,

$$ P(n)=(n-1)+(n-2)+\cdots+1 $$

Using the sum of the first $n-1$ positive integers,

$$ \boxed{P(n)=\frac{n(n-1)}{2}} $$

This formula represents the total number of unique pairs.

4. Pseudocode

```
BRUTE_FORCE_CLOSEST_PAIR(points)
    best_distance = infinity
    best_pair = none
    pair_count = 0
    for i = 0 to n - 2
        for j = i + 1 to n - 1
            pair_count = pair_count + 1
            distance = EuclideanDistance(points[i], points[j])
            if distance < best_distance
                best_distance = distance
                best_pair = (points[i], points[j])
```

```
    return best_pair, best_distance, pair_count
```

5. Python Implementation

```
from math import hypot
from time import perf_counter
def closest_pair_bruteforce(points):
    best_pair = None
    best_distance = float("inf")
    pair_count = 0
    distance_calculations = 0
    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            pair_count +=
            distance = hypot(
                points[i][0] - points[j][0],
                points[i][1] - points[j][1]
            )
            distance_calculations += 1
            if distance < best_distance:
                best_distance = distance
                best_pair = (points[i], points[j])
    return (
        best_pair,
        best_distance,
        pair_count,
        distance_calculations
    )
for n in [10, 100, 500, 1000]:
    points = [(i, (i * 37) % 101) for i in range(n)
    start = perf_counter(
    pair, distance, pairs, calculations = (
```

```
        closest_pair_bruteforce(points)
    )
    elapsed = perf_counter() - start
    print("Points:", n)
    print("Pairs:", pairs)
    print("Distance calculations:", calculations)
    print("Execution time:", round(elapsed, 6))
    print("Closest pair:", pair)
```

OUTPUT:

```
Points: 10
Pairs: 45
Distance calculations: 45
Execution time: 5.3e-05
Closest pair: ((0, 0), (3, 10))
Points: 100
Pairs: 4950
Distance calculations: 4950
Execution time: 0.005401
Closest pair: ((0, 0), (3, 10))
Points: 500
Pairs: 124750
Distance calculations: 124750
Execution time: 0.079563
Closest pair: ((0, 0), (3, 10))
Points: 1000
Pairs: 499500
Distance calculations: 499500
Execution time: 0.323156
Closest pair: ((0, 0), (3, 10))
```

6. Theoretical Results

Number of points	Number of unique pairs
10	45
100	4,950
500	124,750
1,000	499,500
5,000	12,497,500

The number of distance calculations is equal to the number of unique pairs in this implementation.

7. Complexity Analysis

The outer loop processes each point, and the inner loop examines the points that follow it. Therefore, the algorithm performs

$$\frac{n(n-1)}{2}$$

distance calculations.

The time complexity is $O(n^2)$, and the auxiliary space complexity is $O(1)$, excluding the input points.

8. Response-Time Boundary

For this experiment, an acceptable response time can be defined as one second. The program should be executed using increasing input sizes, and the measured time should be recorded.

Number of points	Expected pairs	Measured time	Within one second?
10	45	Record actual time	Yes/No
100	4,950	Record actual time	Yes/No
500	124,750	Record actual time	Yes/No
1,000	499,500	Record actual time	Yes/No
5,000	12,497,500	Record actual time	Yes/No

The first input size that exceeds the chosen time limit indicates the practical boundary for the tested computer and implementation. The boundary may differ on other computers.

9. Conclusion

The Brute-Force Closest Pair algorithm guarantees the correct answer by checking every unique pair. However, the number of calculations increases quadratically with the number of points. It is suitable for small datasets, but large drone fleets may require a more efficient algorithm.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle

1. Problem Context

A wildlife reserve contains several monitoring stations, each represented by a point on a two-dimensional map. Conservation officers want to determine the smallest convex boundary that contains all stations.

This boundary is called the convex hull. It surrounds the input points without bending inward.

2. Geometric Principle

Consider three points:

- $A=(x_1,y_1)$

- $B=(x_2,y_2)$
- $C=(x_3,y_3)$

The orientation of the three points is calculated using the cross product:

$$\operatorname{orientation}(A,B,C) =(x_2-x_1)(y_3-y_1) -(y_2-y_1)(x_3-x_1)$$

The result indicates the position of point C relative to the directed line from A to B.

- A positive result indicates one side of the line.
- A negative result indicates the opposite side.
- Zero indicates that the points are collinear.

If all other points lie on one side of a candidate line, that line may be a supporting line of the convex hull.

3. Pseudocode

```
BRUTE_FORCE_HULL(points)
    edges = empty set
    for every pair A, B of distinct points
        has_positive = false
        has_negative = false
        for every point C other than A and B
            s = orientation(A, B, C)
            if s > 0
                has_positive = true
            if s < 0
                has_negative = true
        if not (has_positive and has_negative)
            record candidate boundary edge A-B
    resolve collinear boundary points
    return boundary vertices in cyclic order
```

4. Python Implementation

```
from math import atan2
def orientation(a, b, c):
    return (
        (b[0] - a[0]) * (c[1] - a[1])
```

```
        - (b[1] - a[1]) * (c[0] - a[0])


def brute_force_hull(points):
    unique = list(dict.fromkeys(points))
    if len(unique) <= 1:
        return unique
    edge_vertices = set()
    for i in range(len(unique)):
        for j in range(i + 1, len(unique)):
            a = unique[i]
            b = unique[j]
            positive = False
            negative = Fals
            for k, c in enumerate(unique):
                if k == i or k == j:
                    continue
                sign = orientation(a, b, c)
                if sign > 0:
                    positive = True
                elif sign < 0:
                    negative = True
                if positive and negative:
                    break
            if not (positive and negative):
                edge_vertices.add(a)
                edge_vertices.add(b)
    if not edge_vertices:
        return unique
    cx = sum(p[0] for p in edge_vertices) / len(edge_vertices)
    cy = sum(p[1] for p in edge_vertices) / len(edge_vertices
```

```
    return sorted(
        edge_vertices,
        key=lambda p: atan2(p[1] - cy, p[0] - cx)

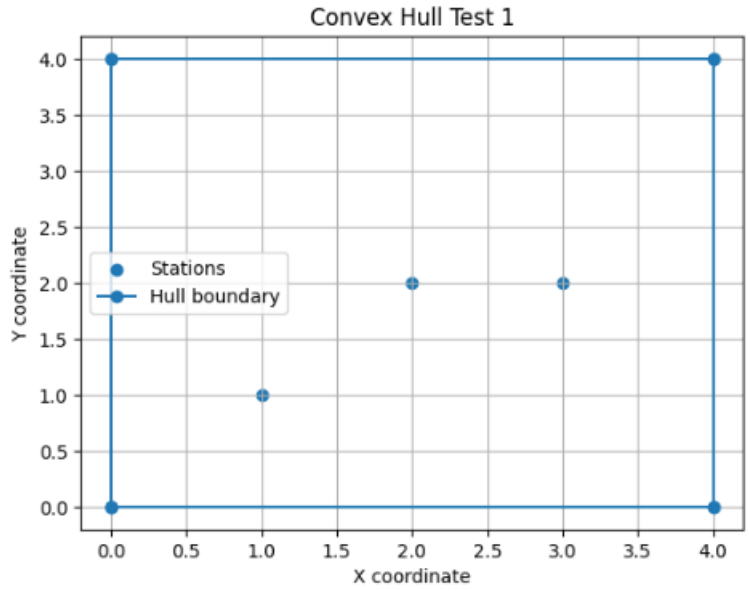
def show_hull(points, hull, title):
    import matplotlib.pyplot as plt
    closed = hull + hull[:1] if len(hull) > 1 else hull
    plt.figure()
    plt.scatter(
        [p[0] for p in points],
        [p[1] for p in points],
        label="Stations"
    )
    if closed:
        plt.plot(
            [p[0] for p in closed],
            [p[1] for p in closed],
            marker="o",
            label="Hull boundary"

    plt.title(title)
    plt.xlabel("X coordinate")
    plt.ylabel("Y coordinate")
    plt.legend()
    plt.grid(True)
    plt.show()
test1 = [
    (0, 0), (4, 0), (4, 4), (0, 4),
    (2, 2), (1, 1), (3, 2)
```

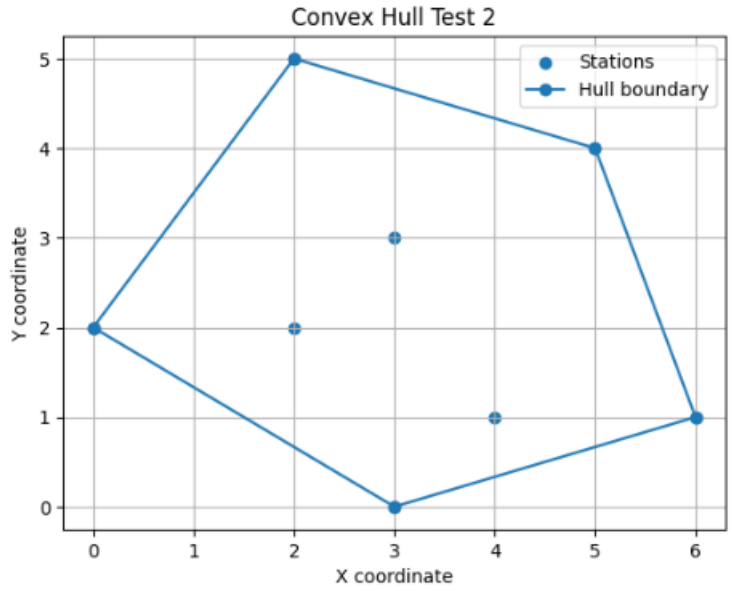
```
test2 = [
    (0, 2), (2, 5), (5, 4), (6, 1),
    (3, 0), (2, 2), (3, 3), (4, 1)
]
for number, points in enumerate([test1, test2], start=1):
    hull = brute_force_hull(points)
    print("Test", number, "Hull:", hull)
    show_hull(points, hull, f"Convex Hull Test {number}")
```

OUTPUT:

```
Test 1 Hull: [(0, 0), (4, 0), (4, 4), (0, 4)]
```




```
Test 2 Hull: [(0, 2), (3, 0), (6, 1), (5, 4), (2, 5)]
```



5. Test Datasets

Test 1: Square with interior points

[(0,0), (4,0), (4,4), (0,4), (2,2), (1,1), (3,2)]

The expected outer boundary consists of the four square corners: \((0,0)\), \((4,0)\), \((4,4)\), and \((0,4)\). The remaining points lie inside the square and should not become outer corners.

Test 2: Irregular point distribution

[(0,2), (2,5), (5,4), (6,1), (3,0), (2,2), (3,3), (4,1)]

This dataset checks the algorithm with an irregular boundary and several interior points. The generated plot should be inspected to confirm that all input points lie inside or on the returned boundary.

6. Complexity Analysis

There are \(O(n^2)\) possible point pairs. For each pair, the algorithm checks up to \(n-2\) other points.

Therefore, the overall time complexity is

\[O(n^2)\times O(n)=O(n^3) \]

The additional storage for the set of boundary vertices is \(O(n)\), excluding the input and plotting structures.

7. Correctness and Limitations

The supporting-line principle is the basis of the algorithm. If points lie on both sides of a candidate line, the line passes through the point set and cannot be an outer supporting line.

However, collinear points require special care. Multiple pairs may lie on the same boundary line. Duplicate points and intermediate points on a straight edge can also affect the reported hull vertices. The implementation should therefore be checked using the supplied datasets and visual plots.

8. Conclusion

The Brute-Force Convex Hull algorithm uses geometric orientation tests to identify candidate boundary edges. Its cubic time complexity makes it appropriate for small datasets, but it becomes expensive when the number of points increases.

Q6 – Exhaustive Search and Combinatorial Growth

1. Problem Context

After a natural disaster, a helicopter must transport emergency supplies to an affected area. Each package has a weight and a humanitarian value. The helicopter has a limited payload capacity.

The objective is to choose a combination of packages that does not exceed the payload capacity while maximising the total humanitarian value.

This problem can be modelled as a 0/1 knapsack problem. Each package is either selected or not selected.

2. Representing Candidate Solutions

Suppose there are \(n\) packages. Each package has two possible states:

- 1 means the package is selected.
- 0 means the package is not selected.

Every combination of these binary decisions represents a possible subset of packages.

Therefore, the total number of possible subsets is

\[2^n \]

The empty subset is also included.

3. Mathematical Growth

Number of packages	Number of subsets
5	32
10	1,024
15	32,768
20	1,048,576
25	33,554,432
30	1,073,741,824

Each additional package doubles the number of subsets. This is exponential growth.

If each subset requires checking up to \(n\) packages, the time complexity of the straightforward implementation is \(O(n2^n)\).

4. Pseudocode

```
EXHAUSTIVE_KNAPSACK(packages, capacity)
    best_value = 0
    best_subset = empty set
    for mask = 0 to 2^n - 1
        total_weight = 0
        total_value = 0
```

```
        chosen = empty set
        for i = 0 to n - 1
            if bit i of mask is 1
                add package i to chosen
                total_weight += weight[i]
                total_value += value[i]
        if total_weight <= capacity
           and total_value > best_value
            best_value = total_value
            best_subset = chosen
    return best_subset, best_value
```

5. Example Package Data

Package	Weight	Humanitarian value
P1	2	12
P2	3	20
P3	4	25
P4	5	30

The helicopter's maximum payload capacity is 7 units.

The combination P2 and P3 has a total weight of

$$ 3+4=7 $$

Its total value is

$$ 20+25=45 $$

The combination P1 and P4 also weighs 7 units, but its value is only

$$ 12+30=42 $$

Therefore, P2 and P3 provide the highest value among these combinations.

6. Python Implementation

```
from time import perf_counter
def exhaustive_knapsack(packages, capacity):
    n = len(packages)
```

```
    best_value = 0
    best_subset = []
    subsets_checked = 0
    feasible_subsets = 0
    for mask in range(1 << n):
        subsets_checked += 1
        total_weight = 0
        total_value = 0
        chosen = []
        for i in range(n):
            if mask & (1 << i):
                package_id, weight, value = packages[i]
                total_weight += weight
                total_value += value
                chosen.append(package_id)
        if total_weight <= capacity:
            feasible_subsets += 1
            if total_value > best_value:
                best_value = total_value
                best_subset = chosen
    return (
        best_subset,
        best_value,
        subsets_checked,
        feasible_subsets
    )
packages = [
    ("P1", 2, 12),
    ("P2", 3, 20),
    ("P3", 4, 25),
```

```
    ("P4", 5, 30)
]
chosen, value, checked, feasible = exhaustive_knapsack(
    packages, 7
)
print("Best packages:", chosen)
print("Total value:", value)
print("Subsets checked:", checked)
print("Feasible subsets:", feasible)
for n in [5, 10, 15, 20]:
    test_packages = [
        (f"P{i+1}", (i % 5) + 1, (i % 9) + 5)
        for i in range(n)
    ]
    start = perf_counter()
    result = exhaustive_knapsack(test_packages, 12)
    elapsed = perf_counter() - start
    print(
        "Packages:", n,
        "Subsets:", result[2],
        "Best value:", result[1],
        "Time:", round(elapsed, 6)
    )
```

OUTPUT:

```
Best packages: ['P2', 'P3']
Total value: 45
Subsets checked: 16
Feasible subsets: 9
Packages: 5 Subsets: 32 Best value: 28 Time: 5e-05
Packages: 10 Subsets: 1024 Best value: 52 Time: 0.002877
Packages: 15 Subsets: 32768 Best value: 57 Time: 0.076196
Packages: 20 Subsets: 1048576 Best value: 70 Time: 3.082037
```

7. Correctness Analysis

The algorithm generates every subset using a bit mask. For each subset, it calculates the total weight and total value.

If the total weight exceeds the capacity, the subset is rejected. If the subset is feasible and has a higher value than the current best solution, the best solution is updated.

Because every subset is examined, the algorithm can find the optimal solution for the given instance.

For four packages, the number of subsets is

\[2^4=16 \]

This small number makes it possible to verify the result manually.

8. Practical Response-Time Boundary

For the experiment, an acceptable response time can be defined as two seconds. The code should be run with increasing numbers of packages, and the actual execution time should be recorded.

Packages	Expected subsets	Measured time	Within two seconds?
5	32	Record actual time	Yes/No
10	1,024	Record actual time	Yes/No
15	32,768	Record actual time	Yes/No
20	1,048,576	Record actual time	Yes/No

The largest tested input that meets the response-time requirement represents the practical boundary for the computer and implementation used in the experiment.

9. Conclusion

Exhaustive Search guarantees an optimal solution when every possible subset is examined correctly. It is suitable for small humanitarian airlift problems where finding the best package combination is important. However, the exponential growth of the search space makes it impractical for large package collections.

Q7 – Diagnose, Redesign and Validate

1. Selected Limitation

The selected limitation is Sequential Search for a growing collection of maintenance-tool records.

Sequential Search is simple and works with unsorted data. However, its execution time increases as the number of records grows. When many maintenance facilities use the same system, repeated searches may cause unnecessary work.

2. Diagnosis of the Computational Bottleneck

The main operation is comparing the requested Tool ID with each record's Tool ID.

If the target is in the first position, only one comparison is needed. If it is in the last position, the algorithm may need \(n\) comparisons. If the key is absent, all \(n\) records must be examined.

For example, if 50,000 searches each examine 100,000 records, the worst-case workload could approach

\[50,000\times100,000 =5,000,000,000 \]

record comparisons.

The main problem is repeated scanning. The same records are checked again and again for different requests.

3. Measurable Design Objective

The objective is to reduce the number of records examined during repeated Tool ID lookups while maintaining correct results when tools are issued or returned.

The proposed target is to replace up to \(n\) record comparisons per linear search with an expected constant-time dictionary lookup after an index has been created.

The hash-based approach is expected to provide \(O(1)\) average lookup time. This is an average-case expectation, not an unconditional guarantee for every possible input.

4. Proposed Redesign: Hash Index

A hash index stores each Tool ID as a key and its corresponding record as a value.

Instead of checking records one by one, the system uses the Tool ID to retrieve the associated record from the dictionary.

The index is built when the records are loaded. It is updated when a tool is issued, returned, or its record changes.

Pseudocode

```
BUILD_INDEX(records)
    index = empty hash table
    for each record in records
        index[record.tool_id] = record
    return index
LOOKUP_TOOL(index, tool_id)
    if tool_id exists in index
        return index[tool_id]
    else
        return NOT FOUND
ISSUE_TOOL(index, record)
```

index[record.tool_id] = recor

RETURN_TOOL(index, tool_id)

remove tool_id from index

5. Preprocessing and Trade-Offs

Factor	**Sequential Search**	**Hash Index**
Preprocessing	Not required	Index must be built
Expected lookup time	\(O(n)\)	\(O(1)\) average
Worst-case lookup	\(O(n)\)	Can degrade with severe collisions
Additional memory	\(O(1)\) for lookup	\(O(n)\) for the index
Unsorted input	Supported	Supported
Updates	Modify records	Keep the index consistent

The hash index requires additional memory and maintenance. Every issue or return operation must update the index correctly.

However, when the same records are searched repeatedly, the initial cost of building the index can be spread across many requests. This makes the redesign suitable for a system supporting multiple maintenance facilities.

6. Prediction Before Implementation

The prediction is that dictionary lookup will reduce the number of application-level lookup operations compared with Sequential Search for large lists.

The difference should become more noticeable as the number of records increases.

However, the index-building time must also be considered. If only one search is performed, building an index may not be beneficial. If many searches are performed, the setup cost is more likely to be worthwhile.

7. Python Implementation and Validation

from time import perf_counte

def linear_lookup(records, key):

comparisons = 0

for record in records:

comparisons +=

if record["tool_id"] == key:

```
            return record, comparison
    return None, comparisons
def build_tool_index(records):
    return {
        record["tool_id"]: record
        for record in records
    }
for n in [100, 1000, 10000, 100000]:
    records = [
        {
            "tool_id": f"T-{i:06d}",
            "technician": f"TECH-{i % 80:03d}"
        }
        for i in range(n)
    ]
    key = f"T-{n-1:06d}"
    start = perf_counter()
    linear_result, comparisons = linear_lookup(records, key)
    linear_time = perf_counter() - start
    start = perf_counter()
    index = build_tool_index(records)
    build_time = perf_counter() - start
    start = perf_counter()
    hash_result = index.get(key)
    hash_time = perf_counter() - star
    print("Records:", n)
    print("Linear comparisons:", comparisons)
    print("Linear time:", round(linear_time, 6))
    print("Index build time:", round(build_time, 6))
    print("Dictionary lookup time:", round(hash_time, 8))
```

```
print("Same result:", linear_result == hash_result)
```

OUTPUT:

```
Records: 100
Linear comparisons: 100
Linear time: 1.2e-05
Index build time: 1.8e-05
Dictionary lookup time: 1.91e-06
Same result: True
Records: 1000
Linear comparisons: 1000
Linear time: 9.5e-05
Index build time: 0.000135
Dictionary lookup time: 1.51e-06
Same result: True
Records: 10000
Linear comparisons: 10000
Linear time: 0.001004
Index build time: 0.001423
Dictionary lookup time: 1.88e-06
Same result: True
Records: 100000
Linear comparisons: 100000
Linear time: 0.009848
Index build time: 0.025742
Dictionary lookup time: 3.47e-06
Same result: True
```

8. Comparison of the Two Approaches

Number of records	Linear comparisons for last record	Dictionary lookup
100	100	One application-level lookup call
1,000	1,000	One application-level lookup call
10,000	10,000	One application-level lookup call
100,000	100,000	One application-level lookup call

The dictionary's internal comparisons are not counted by the application-level code. Therefore, the table compares the number of records examined by Sequential Search with the number of dictionary lookup calls. The measured execution times should be recorded from the actual program output.

9. Results and Discussion

The test deliberately searches for the last record so that Sequential Search performs the maximum number of comparisons. The hash index is built once, and the requested record is then retrieved using its key.

The Same result output verifies whether both methods return the same record. Correctness must be confirmed before claiming that the redesign is successful.

The expected advantage is that dictionary lookup does not require a complete scan of the records. This makes it more suitable for repeated queries over large collections.

10. Final Design Justification

The redesigned approach addresses the identified bottleneck by reducing repeated linear scans. Its expected average lookup complexity is \(O(1)\), compared with \(O(n)\) for Sequential Search.

The redesign also introduces additional memory usage and requires careful maintenance of the index. These trade-offs are reasonable when the system has many records and frequent searches.

Therefore, a hash index is recommended for the expanded maintenance-tool tracking system, provided that the index is maintained correctly as records change.

CONCLUSION

This assignment examined sorting, searching, string matching, computational geometry and exhaustive search. Each algorithm was studied using its working principle, implementation, complexity and practical limitations.

The sorting experiment showed that Insertion Sort is well suited to small, nearly sorted lists. Selection Sort continues to perform quadratic comparisons regardless of the initial order, while Bubble Sort with early stopping can finish quickly on already sorted input.

Sequential Search is suitable for small collections and infrequent queries, but repeated searches over large datasets can become expensive. Brute-Force String Matching demonstrated how text and pattern structure can affect the number of character comparisons.

The Closest Pair algorithm illustrated the quadratic cost of examining every unique pair. The Convex Hull algorithm applied geometric orientation tests to identify candidate boundary edges, with cubic time complexity in the straightforward implementation.

Exhaustive Search demonstrated how every possible package combination can be examined to find the optimal humanitarian airlift solution. Although it guarantees an optimal result for a correctly implemented small instance, its exponential growth limits its practical use for large inputs.

Finally, the Day-4 Design Challenge showed how identifying a computational bottleneck can lead to a practical redesign. Replacing repeated Sequential Search with a hash index reduces the expected lookup cost at the expense of additional memory and index maintenance.

Overall, algorithm selection should consider input size, data arrangement, operation frequency, memory requirements, response-time limits and correctness. Operation counts provide useful evidence for comparing algorithms, while execution-time measurements help determine their practical performance.